AULA 3: Exercício teórico - métodos de ordenação n log n

Aluno: João Marcelo Franca Batista

1) Vetor: {22, 35, 2, 1, 13, 7}

Shellsort (h = 3, 2 e 1)

h = 3: comparo os elementos que estão a 3 posições de distância (posições 0 e 3, 1 e 4, 2 e 5):

22 e 1 -> troca; 35 e 13 -> troca; 2 e 7 -> já estão certos

22 35 2 1 13 7 -> 1 13 2 22 35 7

h = 2: sublistas das posições 0, 2, 4 (1 2 35, já ordenada) e das posições 1, 3, 5 (13 22 7, vira 7 13 22):

1 13 2 22 35 7 -> 1 7 2 13 35 22

h = 1: insertion sort normal:

1 7 2 13 35 22

1 2 7 13 35 22 (o 2 vai para antes do 7)

1 2 7 13 22 35 (o 22 vai para antes do 35)

Resultado: 1 2 7 13 22 35

Quicksort

Pivô escolhido: a mediana entre o primeiro, o do meio e o último elemento de cada parte (mesmo pivô que usei no código da atividade prática).

22 35 2 1 13 7: pivô = 7 (mediana de 22, 2 e 7). Trocas: 22 com 7 e 35 com 1.

Fica: 7 1 2 | 35 13 22

Parte da esquerda (7 1 2): pivô = 2 (mediana de 7, 1 e 2). Troca 7 com 2. Fica: 2 1 | 7

Parte (2 1): pivô = 2. Troca 2 com 1. Fica: 1 | 2

Parte da direita (35 13 22): pivô = 22 (mediana de 35, 13 e 22). Troca 35 com 22. Fica: 22 13 | 35

Parte (22 13): pivô = 22. Troca 22 com 13. Fica: 13 | 22

Juntando tudo: 1 2 7 13 22 35

Heapsort

Primeiro monto o heap máximo (cada pai maior que os filhos). O filho esquerdo da posição i fica em 2i+1 e o direito em 2i+2.

Vetor: 22 35 2 1 13 7

Posição 2 (valor 2) tem filho 7, que é maior: troca -> 22 35 7 1 13 2

Posição 1 (valor 35) tem filhos 1 e 13, que são menores: não muda

Posição 0 (valor 22) tem filho 35, que é maior: troca -> 35 22 7 1 13 2 (o 22 desce, os filhos dele são 1 e 13, então para)

Heap pronto: 35 22 7 1 13 2

Agora troco a raiz com o último elemento do heap e conserto o heap (o que está depois da barra já está ordenado):

troca 35 com 2 -> 2 22 7 1 13 | 35. Conserta: o 2 troca com 22 e depois com 13 -> 22 13 7 1 2 | 35

troca 22 com 2 -> 2 13 7 1 | 22 35. Conserta: o 2 troca com 13 -> 13 2 7 1 | 22 35

troca 13 com 1 -> 1 2 7 | 13 22 35. Conserta: o 1 troca com 7 -> 7 2 1 | 13 22 35
troca 7 com 1 -> 1 2 | 7 13 22 35. Conserta: o 1 troca com 2 -> 2 1 | 7 13 22 35
troca 2 com 1 -> 1 | 2 7 13 22 35

Resultado: 1 2 7 13 22 35

2) Mergesort

Como funciona: o Mergesort divide o vetor no meio, de novo e de novo, até ficarem pedaços de 1 elemento (que já estão ordenados). Depois ele junta (intercala) os pedaços de dois em dois: olha o primeiro elemento de cada pedaço, copia o menor para um vetor auxiliar e continua até acabar um dos pedaços, aí copia o resto do outro. O tempo é n log n em todos os casos, usa um vetor auxiliar (memória extra O(n)) e é estável.

Semelhanças com o Quicksort: os dois usam dividir e conquistar (dividem o vetor em duas partes e ordenam cada uma de forma recursiva) e os dois são n log n no caso médio.

Diferenças:

- O Mergesort sempre divide no meio e o trabalho é feito na hora de juntar. O Quicksort divide usando o pivô (o trabalho é na partição) e as partes podem ficar com tamanhos diferentes.
- Pior caso: Mergesort n log n; Quicksort n^2 (se o pivô for ruim).
- Memória: o Mergesort precisa de um vetor auxiliar; o Quicksort ordena no próprio vetor.
- O Mergesort é estável; o Quicksort não é.

Execução do Mergesort:

22 35 2 1 13 7
divide: [22 35 2] [1 13 7]
divide: [22 35] [2] [1 13] [7]
divide: [22] [35] [2] [1] [13] [7]
junta: [22 35] [2] [1 13] [7]
junta: [2 22 35] [1 7 13]
junta: [1 2 7 13 22 35]

Resultado: 1 2 7 13 22 35

Referências: Cormen et al., Introduction to Algorithms; Sedgewick e Wayne, Algorithms (4ª ed.).